

Podstawy Uczenia Maszynowego

Czym się zajmiemy na ćwiczeniach?

~~Ćwiczenia 1: metody liniowe~~

~~Ćwiczenia 2: metody oparte na sąsiedztwie~~

~~Ćwiczenia 3: metody jądrowe~~

~~Ćwiczenia 4: metody probabilistyczne~~

~~Ćwiczenia 5: klasyfikacja niezbalansowana~~

~~Ćwiczenia 6: selekcja cech i redukcja wymiarowości~~

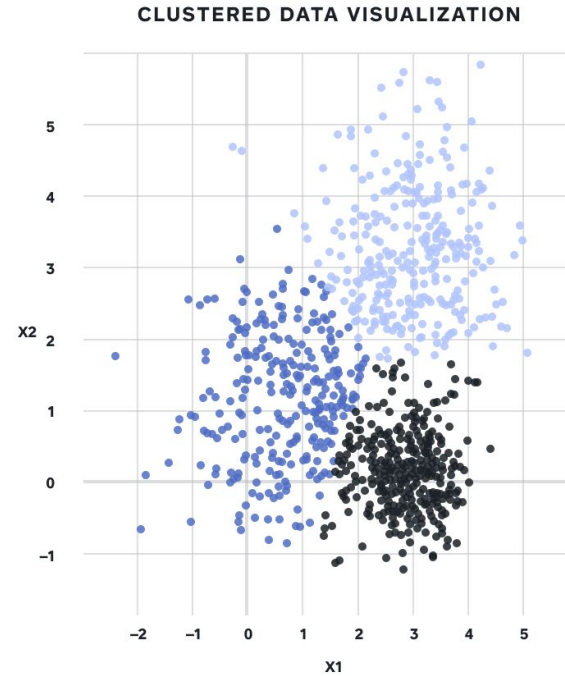
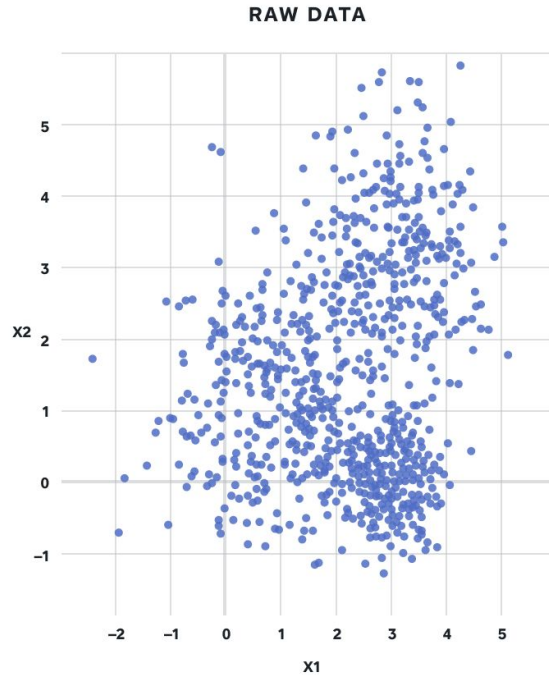
Ćwiczenia 7: klasteryzacja

Klasteryzacja

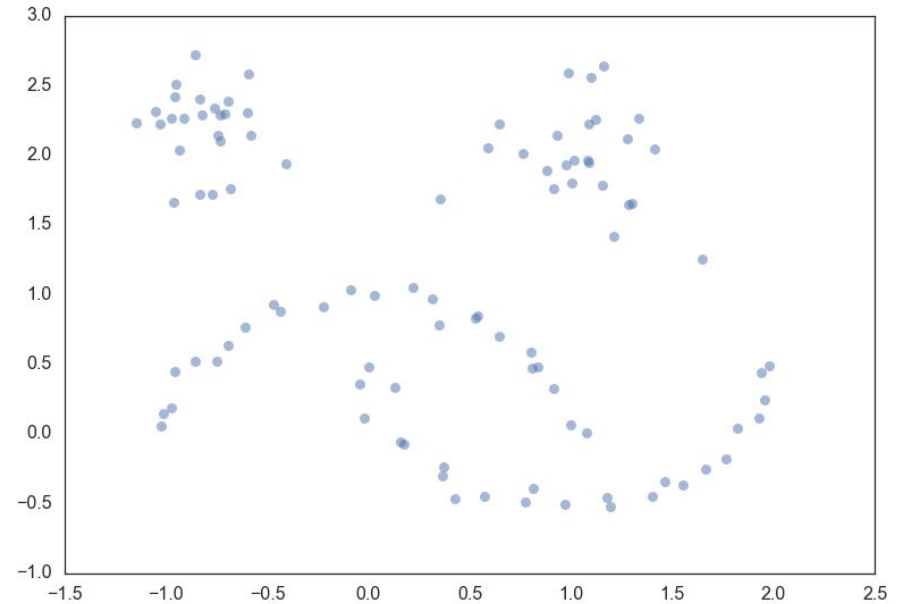
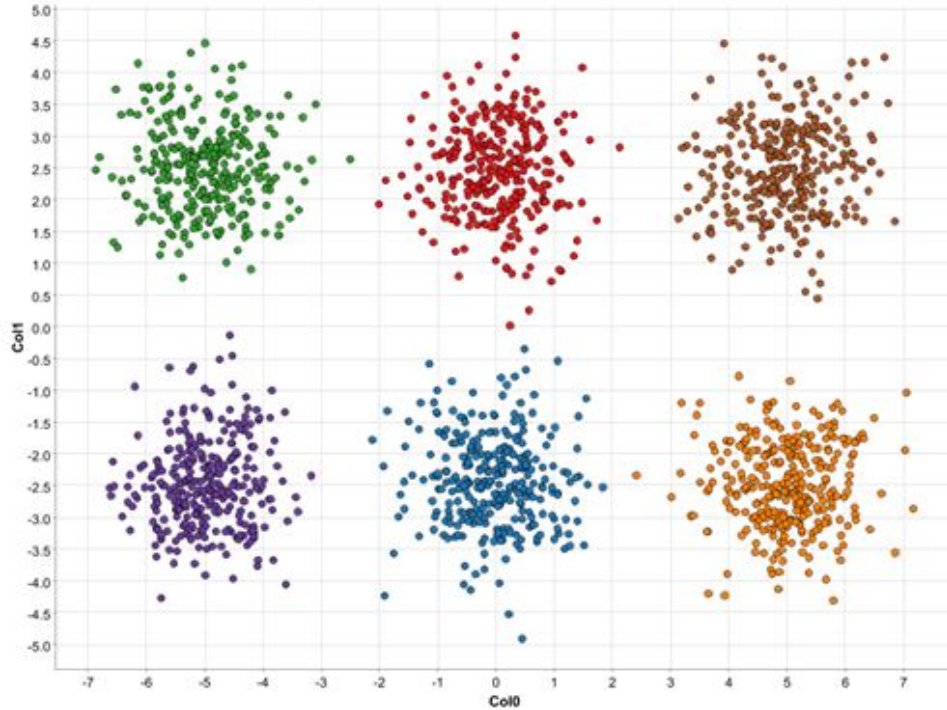
Klasteryzacja (clustering)

- **grupowanie** obiektów, aby odkryć, czy nasze dane mają jakąś strukturę
- typowo w danych występują **klastery (clusters)**, czyli gęściej zbite chmury punktów
- odpowiadają one obiektom o podobnych właściwościach
- mogą, ale nie muszą pokrywać się z klasami:
 - np. zbiór ludzi chorych na nowotwór płuca lub zdrowych
 - rodzaje nowotworu mogą tworzyć klastry wewnątrz klasy pozytywnej
- **klasteryzacja (clustering)** to zadanie odkrycia i analizy takich grup

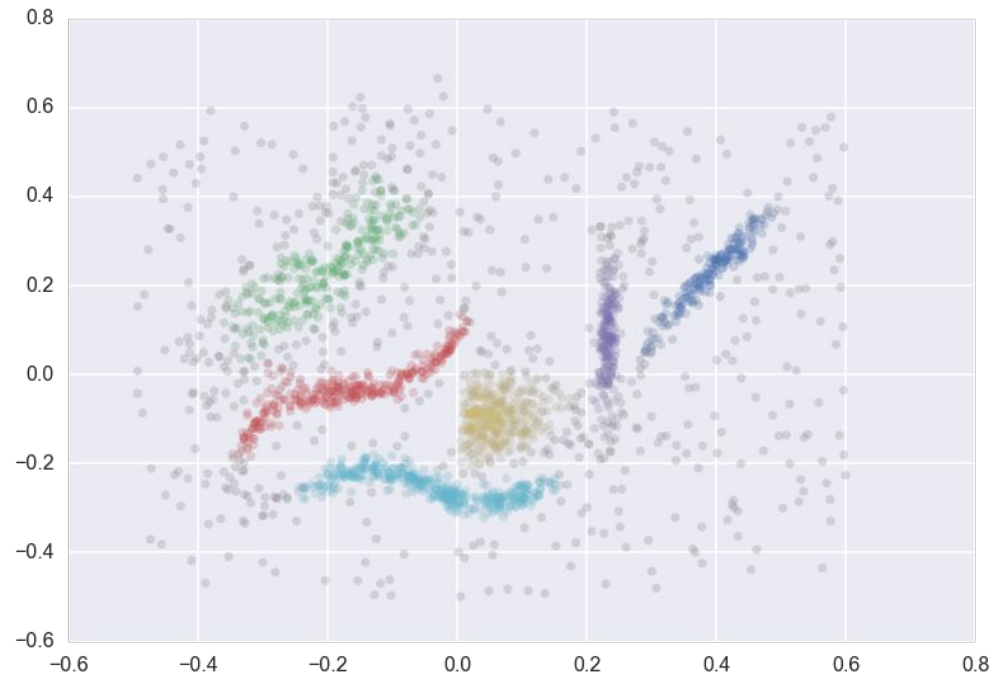
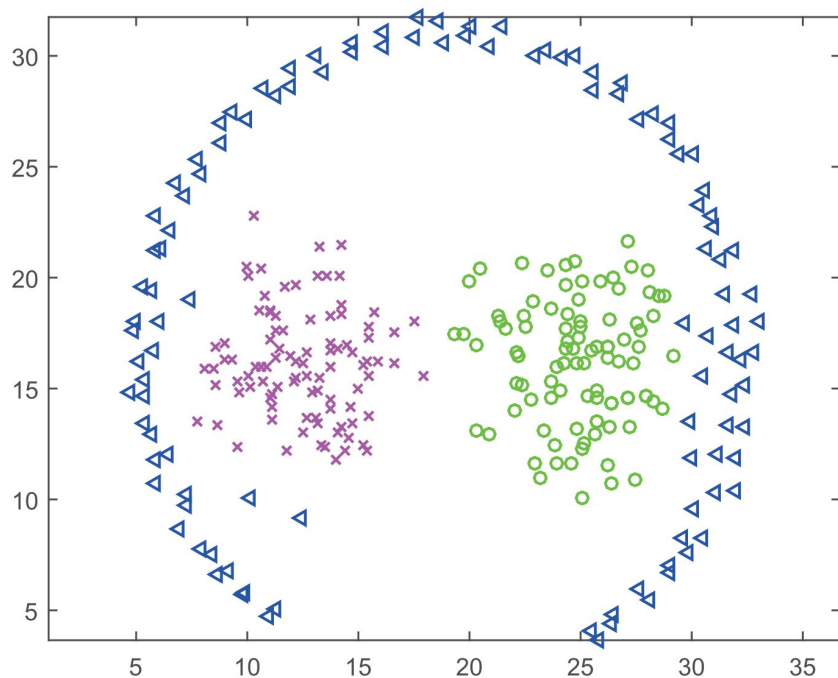
Klasteryzacja



Klasteryzacja prosta (nierealistyczne)



Klasteryzacja trudna (realistyczne)



Zastosowania klasteryzacji

- **analiza zbioru danych:**
 - chcemy zrozumieć lepiej nasze dane i ich strukturę
 - część eksploracyjnej analizy danych (EDA)
 - pozwala np. odkryć potencjalne sub-klasy, albo które klasy będzie najłatwiej mylić
- **segmentacja klientów (customer segmentation):**
 - chcemy odkryć grupy (segmenty) klientów o różnych charakterystykach
 - pozwala np. tworzyć bardziej spersonalizowane kampanie reklamowe

Zastosowania klasteryzacji

- **systemy rekomendacyjne i wyszukiwanie:**
 - istnieje wiele algorytmów rekomendacyjnych opartych o klasteryzację
 - identyfikujemy grupy ludzi o podobnym guście
 - rekomendujemy to, co podobało się innym z tej samej grupy
- **odkrywanie tematów tekstów:**
 - kiedy mamy nowe, duże korpusy tekstów, to nie wiemy, czego dotyczą
 - klasteryzacja pozwala wydajnie odkryć grupy tematyczne tekstów
 - np. analiza recenzji i krytycznych uwag klientów

Zastosowania klasteryzacji

- **biologia ewolucyjna:**

- podstawowe narzędzie biologii ewolucyjnej i bioinformatyki
- pozwala analizować grupy organizmów i ich wspólne pochodzenie
- pozwala tworzyć np. drzewa filogenetyczne

- **genetyka:**

- identyfikacja wspólnie występujących genów (gene coexpression) oraz grup genów
- pozwala znajdować geny i białka o podobnej funkcji, np. do tworzenia nowych leków

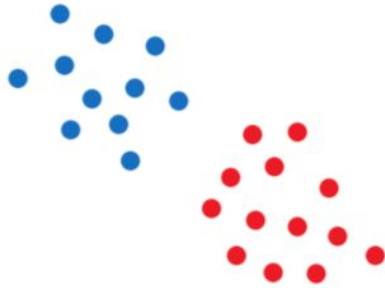
Podejścia do klasteryzacji

- **partitioning / prototype-based / centroid-based:**
 - klaster definiuje jego prototyp, czyli reprezentatywny punkt "na środku" klastra
- **density-based:**
 - klastry to gęste chmury punktów, rozdzielone obszarami rzadkimi
- **connectivity-based / hierarchical / agglomerative:**
 - grupujemy punkty w hierarchię klastrów, łącząc najbliższe w coraz większe klastry
- **inne:**
 - distribution-based, np. Mixture of Gaussians (MoG), Latent Dirichlet Allocation (LDA)
 - oparte o rozkład macierzy, np. Nonnegative Matrix Factorization (NMF)
 - dla big data, np. BIRCH, CLARA, CLARANS

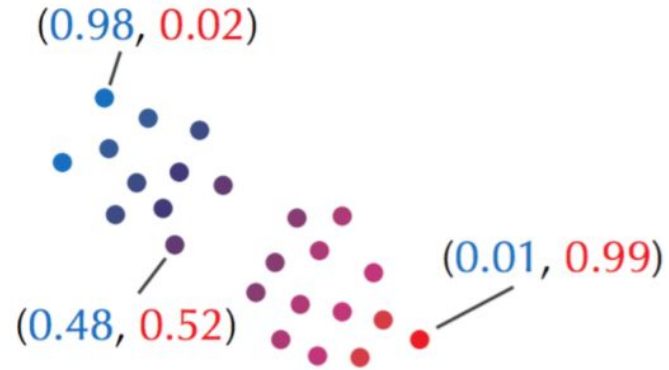
Uwaga co do predykcji

- algorytmy klasteryzacji w większości **nie potrafią** dokonywać predykcji dla nowych punktów
- klastrujemy cały zbiór, a kiedy mamy nowe punkty, trzeba przetrenować algorytm
- **pytanie:** to jest w ogóle sens dzielić na treningowy / walidacyjny / testowy?
- jeżeli nie dokonujemy później żadnej predykcji, to nie
- jeżeli robimy tuning, to potrzebujemy walidacyjnego

Klasteryzacja twarda vs miękka

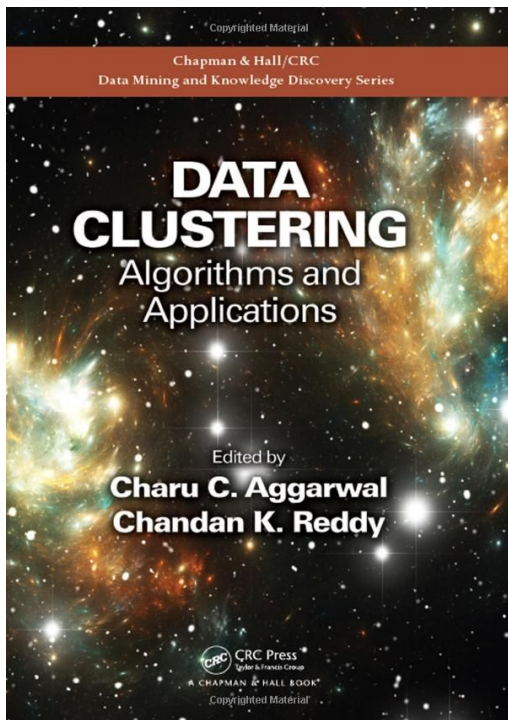


Hard choices: points are colored red or blue depending on their cluster membership.



Soft choices: points are assigned "red" and "blue" *responsibilities* r_{blue} and r_{red} ($r_{\text{blue}} + r_{\text{red}} = 1$)

Źródła



[scikit-learn: clustering](#)
(plus referencje)

k-means

k-means

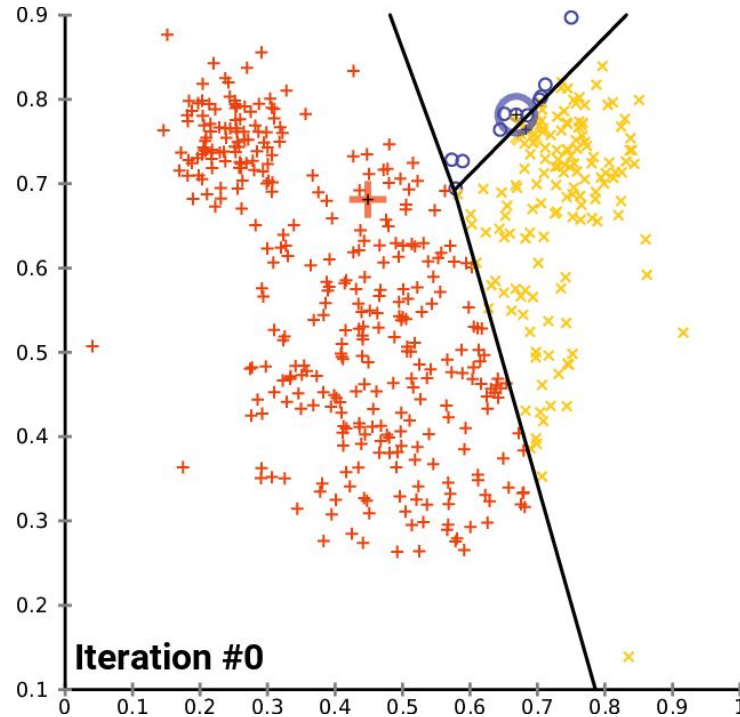
- **partitioning / prototype-based / centroid-based**
- **idea:** klastry powinny być wycentrowane wokół punktów, "reprezentantów" danych grup
- ten punkt to **prototyp (prototype)**, a konkretnie **centroid**, czyli średnia arytmetyczna punktów z klastra
- zakłada, że znamy pożądaną **liczbę klastrów k**
- klasyczna wersja dokonuje twardej klasteryzacji (miękka: fuzzy k-means)

k-means - algorytm

Algorytm:

1. Wybierz początkowe k centroidów, np. jako losowe punkty ze zbioru
 2. Powtarzaj aż do osiągnięcia zbieżności:
 - a. Assignment: przyporządkuj każdy punkt do najbliższego centroidu
 - b. Update: zaktualizuj koordynaty centroidów jako średnią z przypisanych do niego punktów
- zbieżność to typowo mała zmiana w koordynatach centroidów, plus limit iteracji
 - tzw. algorytm Lloyd'a

k-means - wizualizacja



k-means - formalnie

- **założenia:**
 - klastry mają **rozkład normalny**, wycentrowany na centroidach
 - co więcej, rozkłady te są **sferyczne (izotoniczne)** - zerowa kowariancja
- taki klaster jednoznacznie definiuje jego centroid, czyli wartość średnia
- **chcemy:**
 - **skoncentrowane** klastry - o małej wariancji
 - **dobrze rozdzielone** klastry - o wyraźnej separacji (odległości)

k-means - formalnie

- k-means minimalizuje **wariancję klastrów (within-cluster sum of squares, WCSS)**:

$$\arg \min_S \sum_{i=1}^k \sum_{x \in S_i} ||x - \mu_i||^2 = \arg \min_S \sum_{i=1}^k |S_i| \text{Var}(S_i)$$

- nazywane też inercją (inertia)
- jest to równoważne **minimalizacji wariancji** wewnątrz klastrów
- z **prawa całkowitej wariancji (law of total variance)** jest to równoważne maksymalizacji wariancji pomiędzy klastrami

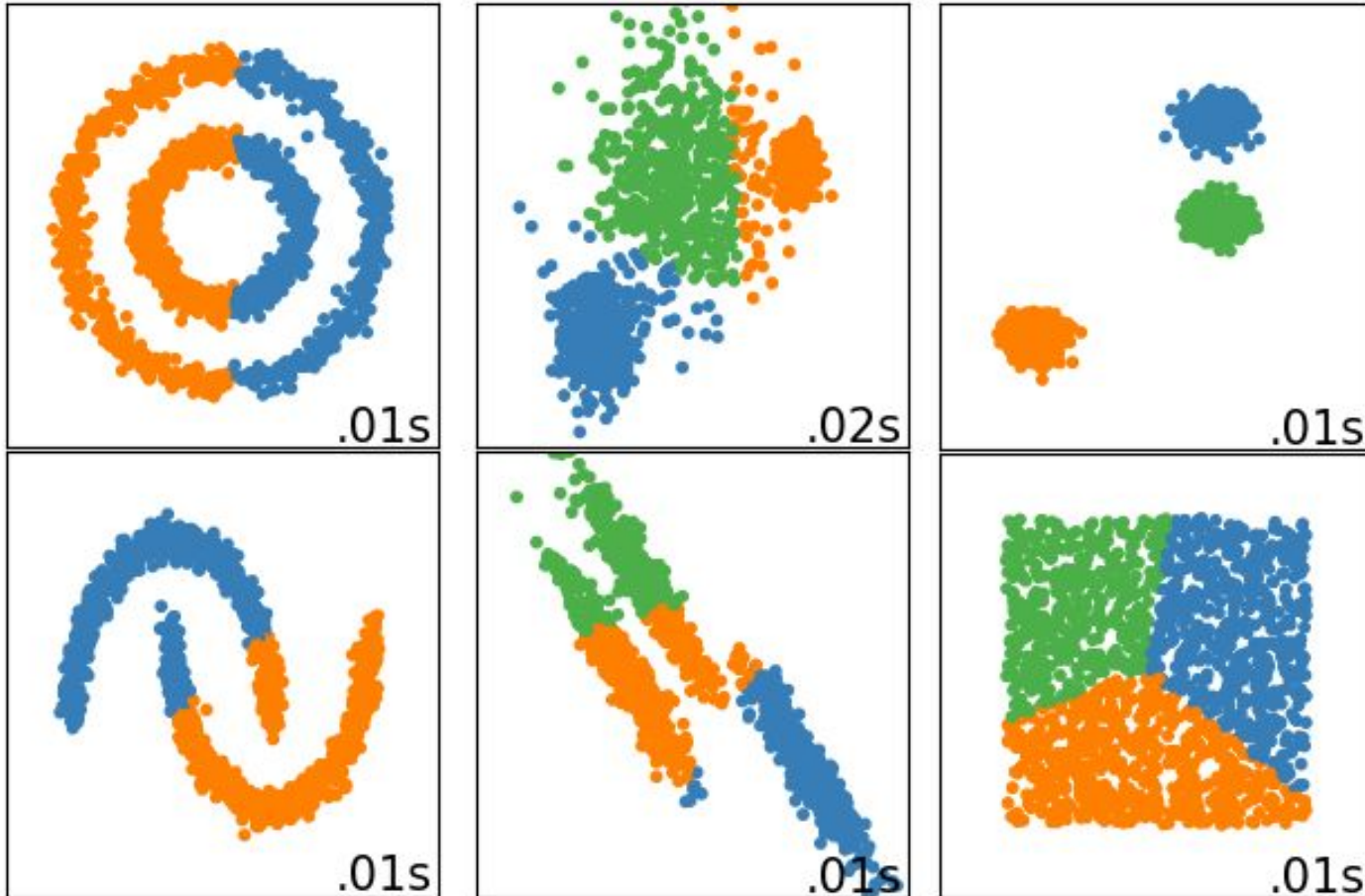
k-means - metryka

- k-means ma więc **funkcję kosztu** - de facto algorytm minimalizuje ją według pochodnej
- **implicite** zakładamy odległość euklidesową, która jest zawarta w definicji wariancji
- **tylko** ona działa w połączeniu ze średnią, inaczej nie mamy gwarancji zbieżności
- **alternatywy:**
 - k-medians: metryka L1 + mediana
 - k-modes (dane katégoryczne): simple match metric + moda

k-means - konsekwencje założeń

- k-means wymaga **normalizacji** danych, bo liczymy odległości
- występuje **klątwa wymiarowości**, typowo zwalczana z pomocą PCA
- jest **czułe na outliery**, główna wada algorytmu
- taka funkcja kosztu **preferuje klastry podobnej wielkości**:
 - w sensie objętości przestrzeni, nie liczby punktów
 - wtedy wariancja się rozkłada się równo między klastry

k-means - konsekwencje założeń



k-means - inicjalizacja

- minimalizujemy funkcję kosztu, ale **algorytmem zachłannym**
- trafiamy do **minimum lokalnego**
- **co z tym zrobić?**
 - uruchomić wielokrotnie i wybrać klasteryzację z minimalnym WCSS
 - lepiej inicjalizować - algorytmem **k-means++**

K-means++

- **chcemy:** dobrze rozdzielonych klastrów
- **obserwacja:** k-means przesuwa centroidy, zaczynając od punktów początkowych
- **idea:**
 - klastry powinny być dobrze rozdzielone
 - to "rozrzućmy" ich początkowe lokalizacje

k-means++ - algorytm

1. Wylosuj pierwszy centroid
 2. Powtarzaj do uzyskania k centroidów:
 - a. Dla każdego z punktów poza centroidami oblicz $D(x)$, odległość między x a najbliższym centroidem
 - b. Wylosuj nowy centroid, przy czym punkt x jest wybierany z prawdopodobieństwem proporcjonalnym do $D^2(x)$ (rozkład prawd. ważony odległościami)
- wybiera centroidy, które są "średnio dość daleko" od już wybranych

K-means++

- **znacznie** lepsza jakość klasteryzacji (szczególnie dla wielu klastrów)
- paradoksalnie **skraca czas wykonania**, bo o ile wymaga czasu, to potem osiągnięcie zbieżności jest dużo szybsze
- domyślna opcja w Scikit-learn
- **w praktyce:**
 - zawsze używamy K-means++, a do tego odpalamy algorytm kilka razy i bierzemy najlepszy wynik (n_init)

k-means - hiperparametry

- **liczba klastrów k:**
 - najważniejszy hiperparametr
 - **ciężko** dobrać - nie można użyć WCSS, bo im więcej klastrów, tym mniejsze mają wariancje!
 - są do tego metryki, ale jest to ogólnie trudne, i wymaga interpretacji klastrów
- są inne, ale mają małe znaczenie

k-means - predykcja

- algorytmy klasteryzacji oparte o prototypy **potrafią** dokonywać predykcji
- klaster dla nowego punktu to najbliższy prototyp, np. centroid

k-means - koszt obliczeniowy

- predykcja klastra dla nowego punktu to znalezienie najbliższego sąsiada wśród centroidów
- **złożoność treningu:** $O(ndk * i)$
i - liczba iteracji, typowo kilkaset
k - liczba klastrów
- **złożoność predykcji:** $O(ndk)$

k-means - zalety i wady

Zalety:

- prostota
- bardzo niski koszt obliczeniowy
- doskonale uwspółbieżnialne
- interpretowalność (można sprawdzić punkty najbliższej centroidu)
- potrafi robić predykcję

Wady:

- bardzo mocne założenie klastrów sferycznych
- prowadzi do klastrów podobnych rozmiarów
- ciężko dobrać liczbę klastrów
- tylko metryka euklidesowa
- czułe na outliery
- centroid to nieistniejący punkt

Metryki w klasteryzacji

Problem z metrykami do klasteryzacji

- jeżeli zdefiniujemy klastry **tylko** jako "zbite" czy "gęste", to więcej = lepiej
- prowadziłoby do liczby klastrow równej liczbie próbek - **nie działa!**
- **metryka musi:**
 - chcieć dobrze zdefiniowanych i odseparowanych klastrow
 - nie preferować zawsze większej liczby klastrow - być **niemonotoniczna**
 - być nienadzorowana, oceniać klastrowanie "samo w sobie" - tzw. **internal measure**
- **uwaga:** metryki nienadzorowane często liczymy na całym zbiorze, bez podziału na treningowy i testowy

Uwaga co do porównywania algorytmów

- porównywanie algorytmów klasteryzacji jest **bardzo trudne**
- metryki generalnie **preferują klastry wypukłe**, co dyskryminuje algorytmy klastrujące dowolne kształty, np. DBSCAN
- trzeba dokładnie **analizować klastry**, czy mają sens
- stosowanie metryk w obrębie jednego algorytmu, np. do wyboru liczby klastrów, jest generalnie ok

Silhouette score

- **silhouette coefficient** dla pojedynczego punktu:

$$\frac{b - a}{\max(a, b)}$$

a - średnia odległość do pozostałych punktów w klastrze

b - średnia odległość do punktu z innego, najbliższego klastra

- **silhouette score** to średnia wartość dla całego zbioru
- miara tego, jak bardzo klaster jest "zbity", względem tego, jak bardzo jest oddalony od najbliższego innego klastra

Silhouette score - zalety i wady

Zalety:

- interpretowalne
- dobre wyniki
- zakres wartości $[-1, 1]$
- dowolna metryka

Wady:

- preferuje klastry wypukłe
- koszt obliczeniowy $O(n^2)$

Calinski-Harabasz Index

- WCSS - within-cluster sum of squares
- BCSS - between clusters sum of squares
- μ - globalna średnia (centroid) całego zbioru
- μ_i - centroid i-tego klastra
- n_i - liczba elementów w i-tym klastrze
- **Calinski-Harabasz index** to stosunek tego, jak dobrze rozdzielone są klastry (BCSS), względem tego, jak dobrze zbite są (WCSS)
- **zakres wartości:** $[0, +\infty)$, większe wartości są lepsze
- wartości zależą od struktury i wielkości zbioru, więc CH index jest nieporównywalny między zbiorami

$$WCSS = \sum_{i=1}^k \sum_{x \in S_i} ||\mu_i - x||^2$$

$$BCSS = \sum_{i=1}^k n_i ||\mu - \mu_i||^2$$

$$CH = \frac{BCSS}{k - 1} / \frac{WCSS}{n - k}$$

Calinski–Harabasz index - zalety i wady

Zalety:

- prostota
- intuicyjne sformułowanie
- szybkie - złożoność $O(n)$

Wady:

- preferuje klastry sferyczne
- trudny w interpretacji zakres wartości
- czuły na szum
- tylko metryka euklidesowa

Inne metryki

- **cophenetic correlation coefficient (CPCC):**
 - dla klasteryzacji hierarchicznej
 - uwzględnia stabilność klastrów w hierarchii
- **density-based cluster validation (DBCV):**
 - dla klasteryzacji opartej o gęstość, np. DBSCAN, HDBSCAN
 - uwzględnia fakt, że klastry typowo nie są wypukłe

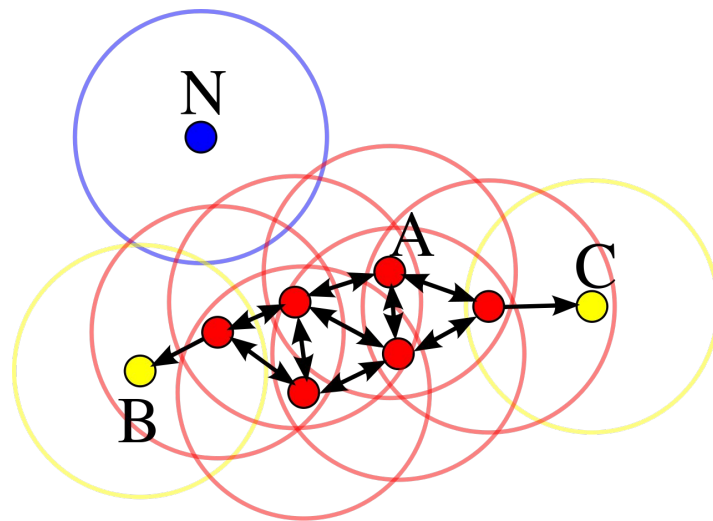
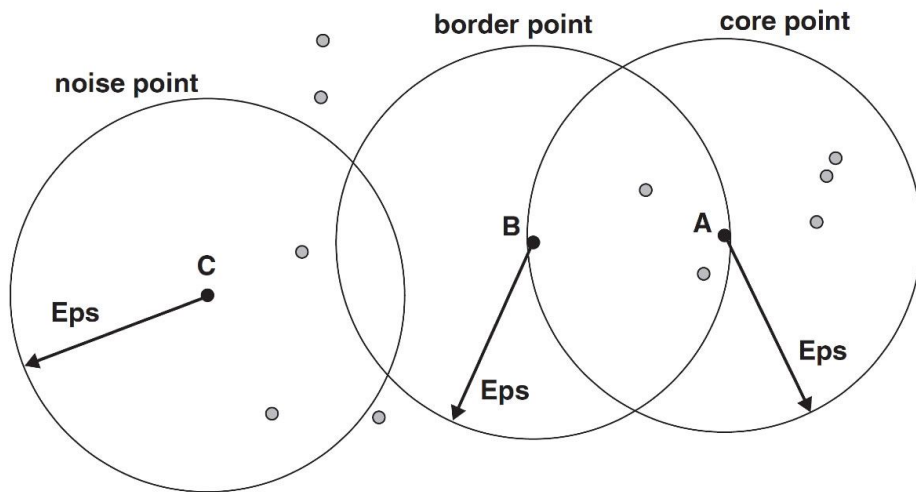
DBSCAN

DBSCAN

- **density-based**
- **Density-Based Spatial Clustering for Applications with Noise**
- **idea:**
 - klastry to gęste obszary, rozdzielone rzadkimi obszarami
 - nie każdy punkt musi być w klastrze, zawsze jest jakiś szum
- odpowiada **oszacowaniu rozkładu prawdopodobieństwa** - klastry to obszary, gdzie prawd. punktów jest większe

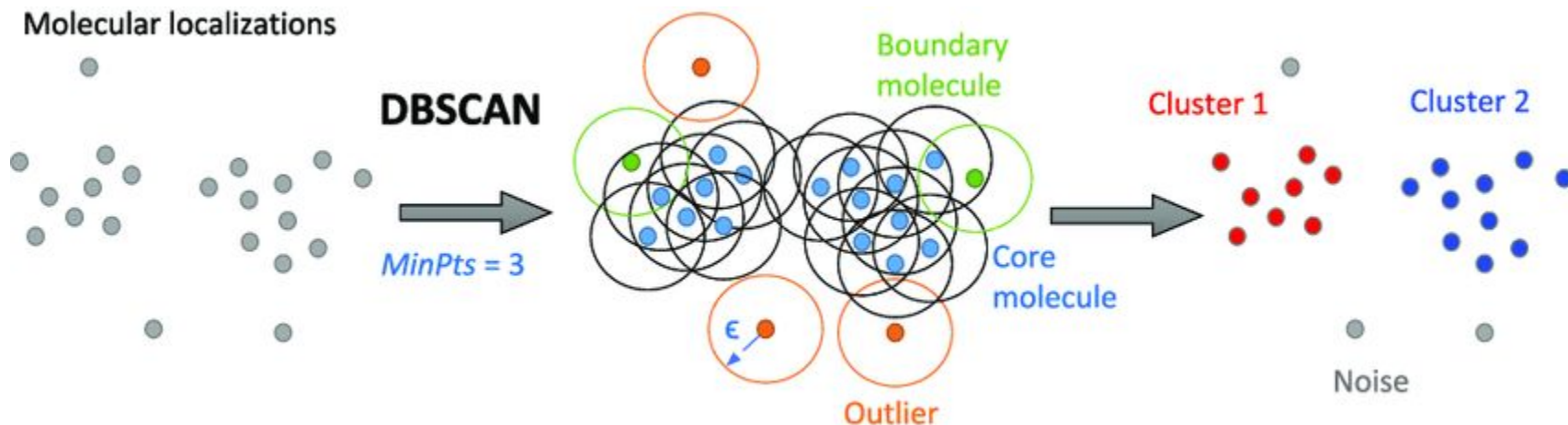
DBSCAN - algorytm

1. Dla każdego punktu znajdź jego najbliższych sąsiadów w **promieniu epsilon**
2. Oznacz punkty:
 - sąsiedztwo ma co najmniej **min_pts punktów** - punkt CORE (obszar gęsty)
 - mniej punktów, ale sąsiadem jest punkt CORE - punkt BORDER (obok obszaru gęstego)
 - w innym wypadku - punkt NOISE

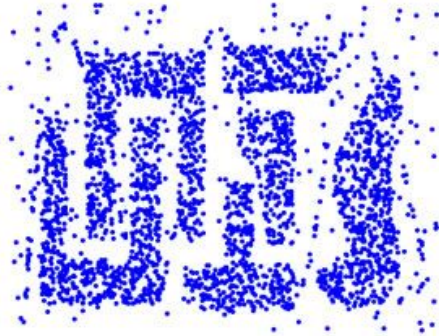


DBSCAN - algorytm

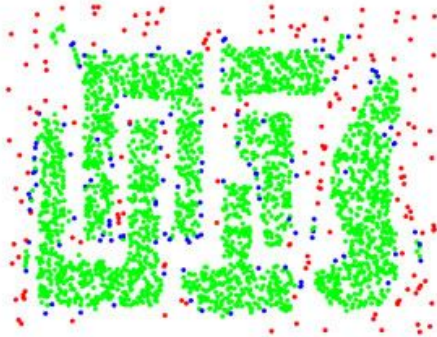
3. Stwórz klastry, łącząc sąsiednie punkty CORE
4. Dołącz punkty BORDER do sąsiedniego klastra (losowo w przypadku remisów)



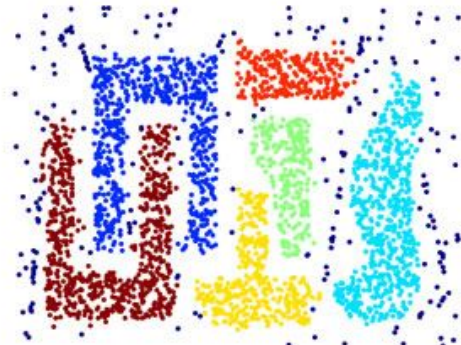
DBSCAN - wizualizacja



Original Points



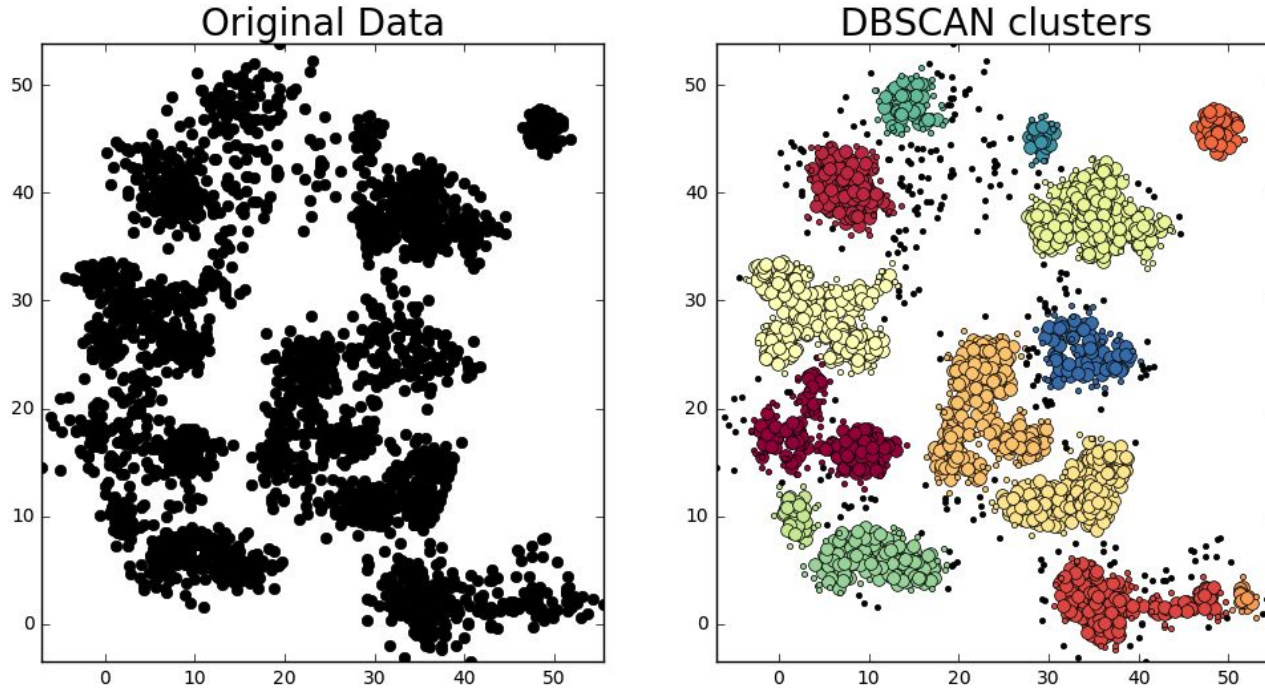
Point types: **core**,
border and **noise**



Clusters

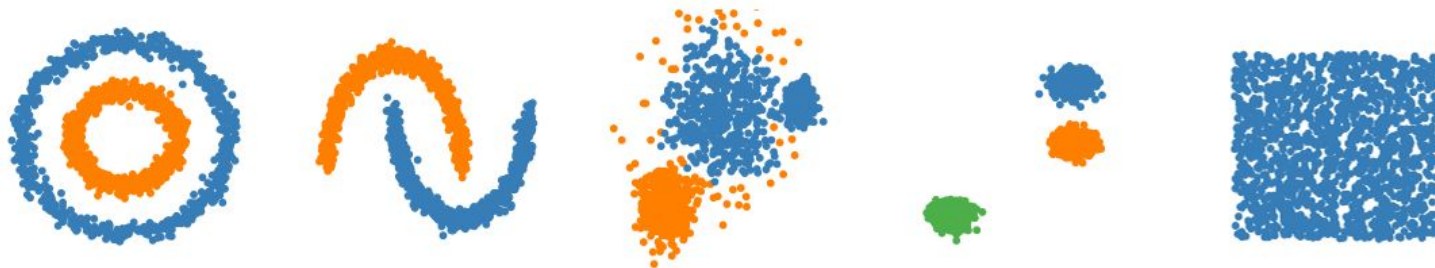
Eps = 10, MinPts = 4

DBSCAN - wizualizacja

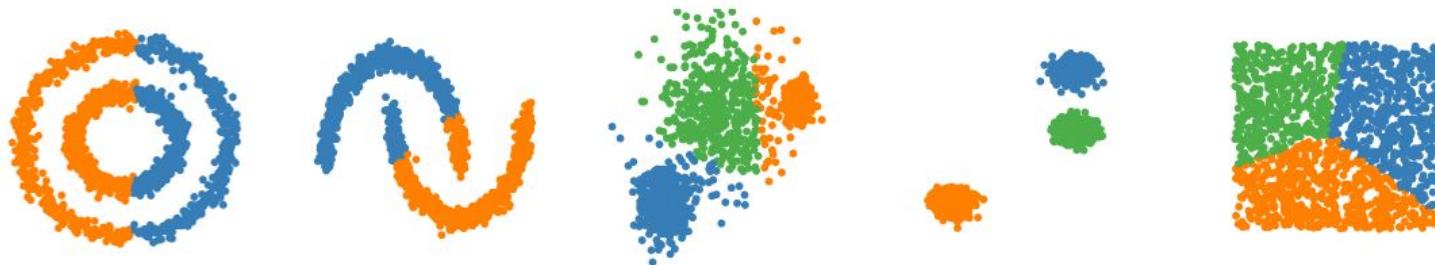


DBSCAN vs k-means

DBSCAN



k-means



DBSCAN - własności

- **sam wykrywa klastry**, ich liczba zależy od promienia epsilon i min_pts
- klastry mogą być **dowolnych kształtów**
- **czuły na hiperparametry** i typowo ciężko je dobrać:
 - oryginalnie do danych przestrzennych (spatial), więc epsilon był np. w metrach
 - epsilon: ciężko dobrać, można analizować rozkład odległości w zbiorze
 - min_pts: typowo 1-2 razy tyle, ile wymiarowość danych
- **kiepski przy zróżnicowanej gęstości**, bo epsilon i min_pts są jedne, globalne
- przez powyższe potrafi być też **podatny na szum**, szczególnie przy źle dobranych hiperparametrach
- **klątwa wymiarowości**, bo sprawdzamy najbliższych sąsiadów

DBSCAN - złożoność obliczeniowa

- najbliżsi sąsiedzi dla n punktów, z pomocą jakiejś struktury indeksującej
- **złożoność obliczeniowa:** $O(n \log(n))$
- **złożoność pamięciowa:** $O(n)$
- w praktyce mocno zależy od wymiarowości

DBSCAN - zalety i wady

Zalety:

- prostota
- szybkość i skalowalność
- klastry o dowolnym kształcie
- dowolna metryka odległości

Wady:

- ciężko dobrać hiperparametry
- kiepski przy zróżnicowanej gęstości
- kłątwa wymiarowości
- potrafi być podatny na szum

HDBSCAN

Co warto ulepszyć w DBSCAN?

- **punkty BORDER:**
 - dodawanie ich jest mocno heurystyczne, pogarsza interpretację statystyczną
 - potrafi być niedeterministyczne przy blisko położonych klastrach
 - zamiana BORDER na NOISE -> **DBSCAN***
- **uwzględnienie zróżnicowanej gęstości:**
 - założenie stałego epsilon to główny powód podatności na szum
 - adaptacyjne sprawdzenie różnych wartości epsilon -> **OPTICS**

Co warto ulepszyć w DBSCAN?

- **pozbycie się hiperparametru epsilon:**
 - ten hiperparametr ciężko dobrać w arbitralnych przestrzeniach
 - trudne - wymaga, żeby epsilon "sam się wybrał", i to adaptacyjnie
 - ale da się - **HDBSCAN**

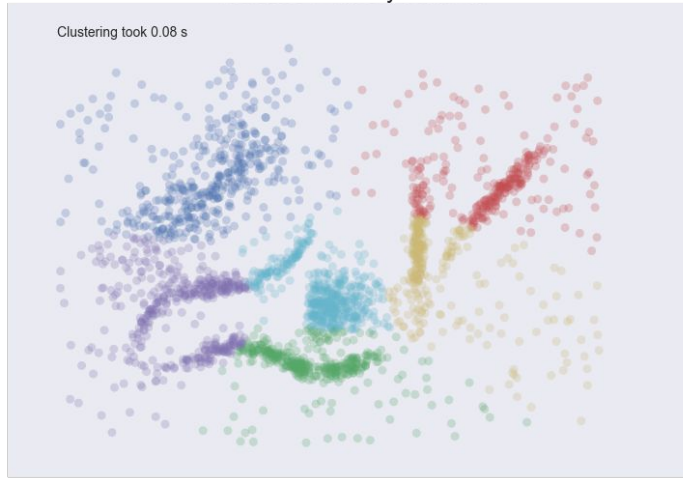
HDBSCAN*

- **Hierarchical DBSCAN***
- łączy:
 - DBSCAN - główna idea
 - DBSCAN* - brak punktów BORDER
 - klasteryzacja hierarchiczna - drzewo klastrów, sprawdzające różne wartości epsilon
- **jeden z najlepszych** algorytmów klasteryzacji

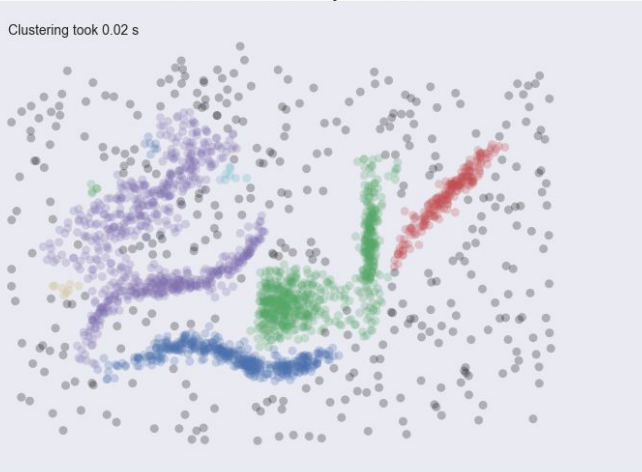
HDBSCAN* - zalety

- **bardziej odporny na szum** od DBSCAN
- **szybszy** niż DBSCAN:
 - pomimo sprawdzania wielu wartości epsilon!
 - ulepszenia algorytmiczne + doskonała implementacja
 - **złożoność:** $O(n \log(n))$
- **proste hiperparametry:**
 - min_cluster_size - minimalna wielkość klastra
 - min_pts - jak w DBSCAN, opcjonalne

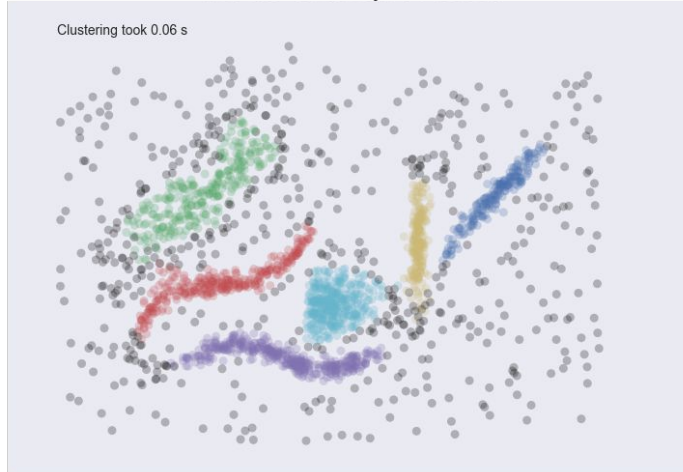
Clusters found by KMeans



Clusters found by DBSCAN

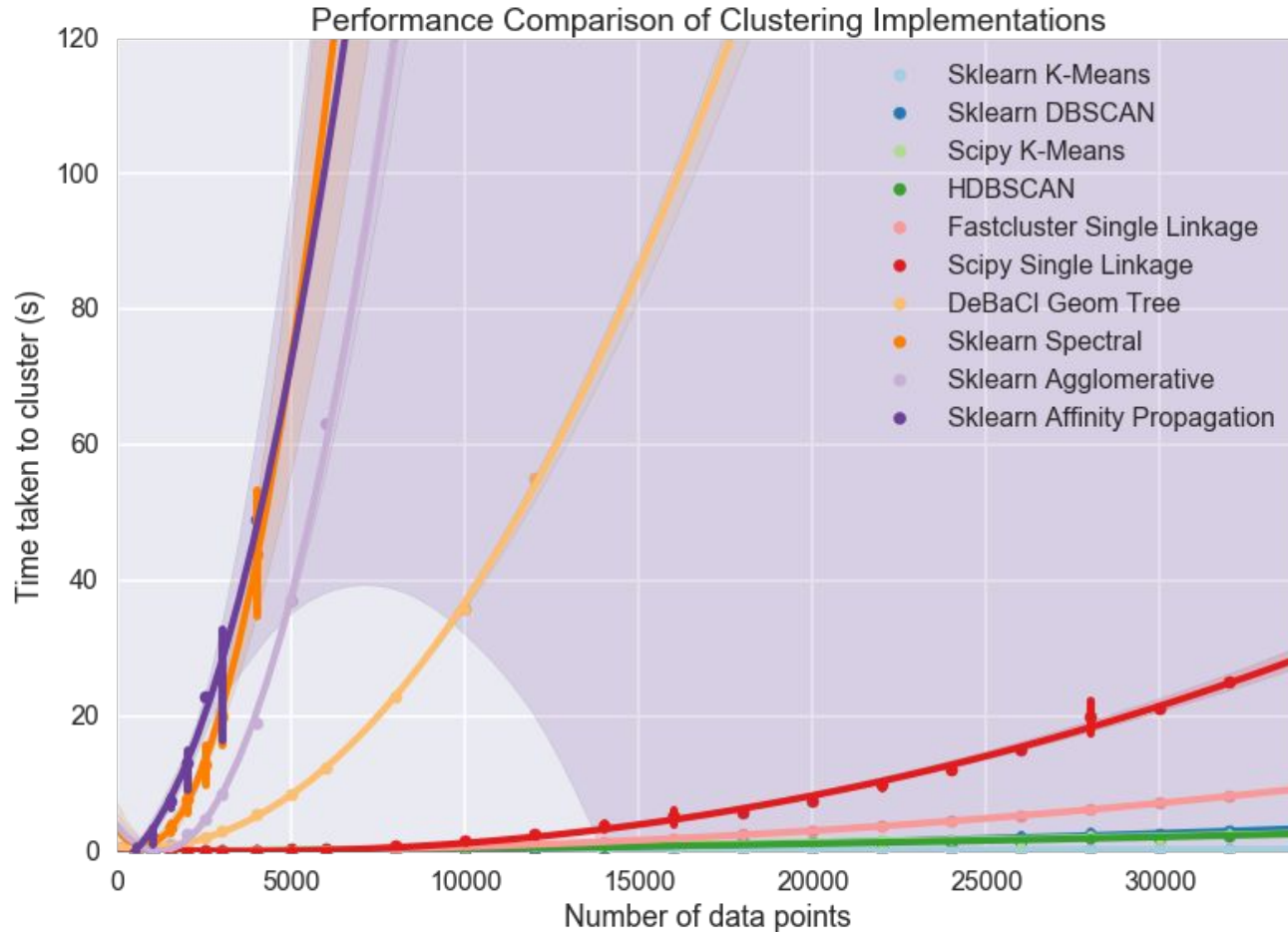


Clusters found by HDBSCAN



HDBSCAN*
vs inne
algorytmy

HDBSCAN* - skalowalność



HDBSCAN* - źródła

- [HDBSCAN, Fast Density Based Clustering, the How and the Why - John Healy](#)
- [High Quality, High Performance Clustering with HDBSCAN - L. McInnes](#)
- ["Accelerated Hierarchical Density Clustering" L. McInnes, J. Healy](#)

Do diety!

Slajdy bonusowe

Klasteryzacja hierarchiczna

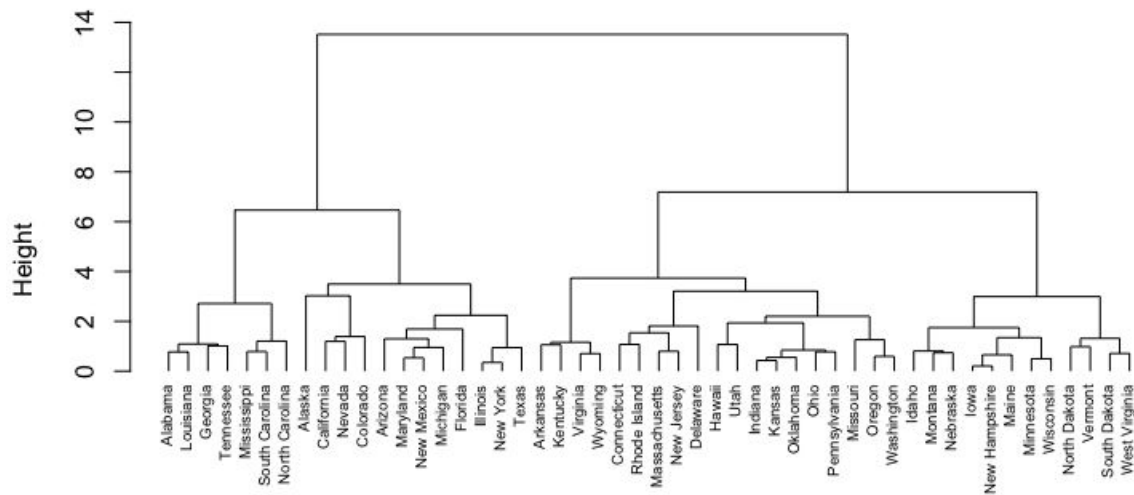
Klasteryzacja hierarchiczna

- **connectivity-based / hierarchical / agglomerative**
- **idea:**
 - klastry tworzą punkty blisko siebie
 - zbiór może mieć wiele klastrów, o różnych rozmiarach, ale wszystkie powinny być gęste
 - duże klastry mogą się składać z mniejszych
- metody hierarchiczne tworzą **hierarchię klastrów**, łącząc mniejsze klastry w coraz większe
- trzeba wybrać, ile finalnie klastrów chcemy

Klasteryzacja hierarchiczna - algorytm

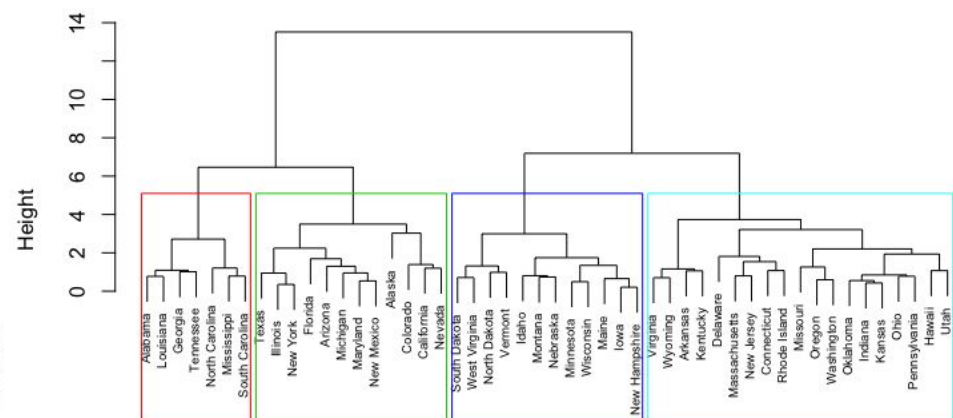
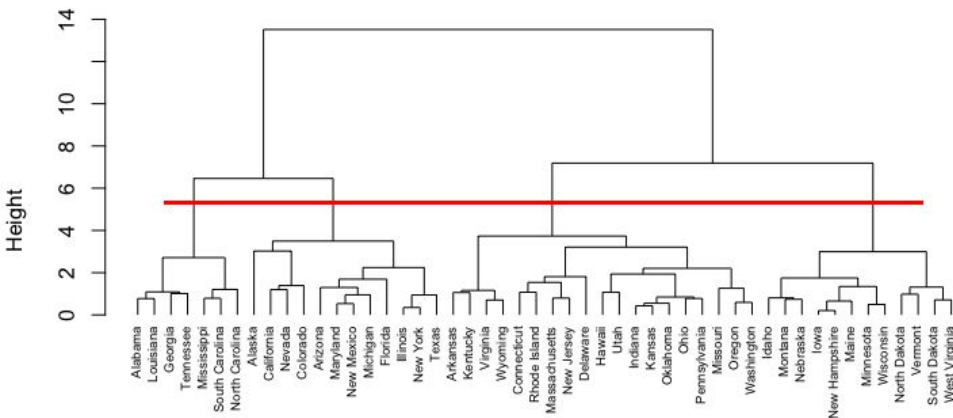
1. Inicjalizacja: każdy punkt to 1-elementowy klaster
 2. Powtarzaj do uzyskania jednego klastra:
 - a. Oblicz odległości między klastrami (każdy-z-każdym)
 - b. Znajdź 2 najbliższe klastry
 - c. Połącz je
- jest to typowo używany algorytm **agglomerative (bottom-up)**, zachłanny
 - dodatkowo zapamiętuje się odległości między klastrami, żeby narysować **dendrogram**

Klasteryzacja hierarchiczna - dendrogram

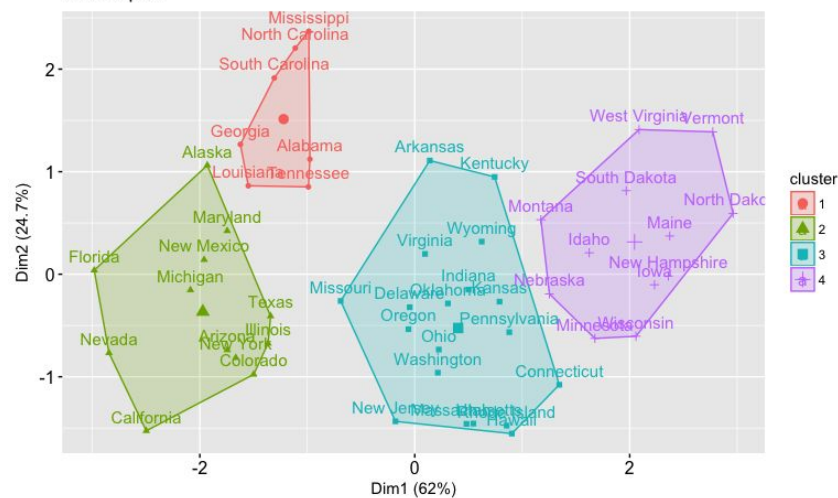


- **długość linii** odpowiada odległości między klastrami
- **długie** = stabilne, dobrze rozdzielone klastry
- **płaskie klastrowanie (flat clustering)** odpowiada wybraniu "poziomu"
- typowo przecinamy tam, gdzie linie są długie, i gdzie widać "grupy" poniżej

Klasteryzacja hierarchiczna - dendrogram



Cluster plot



Jak łączyć klastry?

- można różnie identyfikować, **co znaczy "najbliższe" klastry**
- 4 główne sposoby:
 - single linkage
 - complete (maxium) linkage
 - average linkage
 - Ward linkage

Jak łączyć klastry?

Single Linkage:

- odległość między najbliższymi punktami z obu klastrów
- **zalety:** szybkość, dobrze wykrywa klastry o złożonych kształtach
- **wada:** bardzo podatne na szum (tzw. cluster chaining)

Complete (maximum) Linkage:

- odległość między najdalszymi punktami z obu klastrów
- odpowiada łączeniu klastrów, które łącznie są mocno zbite (mają mały promień)
- **zalety:** mocno skoncentrowane klastry
- **wada:** podatne na szum, preferuje klastry o podobnych rozmiarach

Jak łączyć klastry?

Average Linkage:

- średnia odległość między punktami z obu klastrów
- **zalety:** odporne na szum
- **wada:** preferuje klastry sferyczne, koszt obliczeniowy

Ward Linkage:

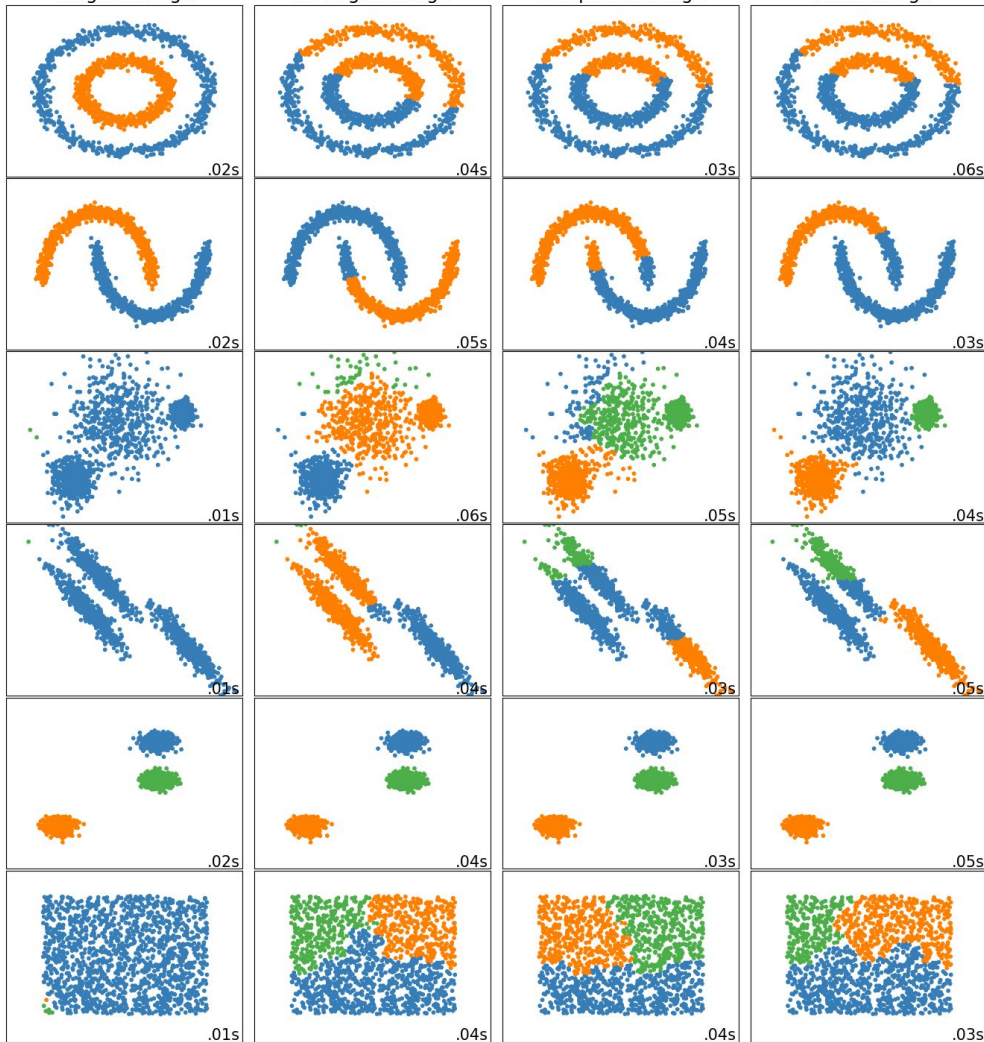
- obliczamy SSE (kwadraty odległości z klastra do centroidu) dla klastrów
- łączymy tak, aby po połączeniu SSE (do nowego centroidu) wzrosło najmniej
- **zalety:** bardzo odporne na szum
- **wada:** preferuje klastry sferyczne, koszt obliczeniowy

Single Linkage

Average Linkage

Complete Linkage

Ward Linkage



Sposoby łączenia klastrow - wizualizacja

Kl. hierarchiczna - złożoność obliczeniowa

- odległości każdy do każdego - $O(n^2)$
- powtarzamy to w pętli dla n punktów - $O(n)$
- **złożoność treningu:** $O(n^3)$
- **złożoność pamięciowa:** $O(n^2)$
- używając **kopca (heap)** do wyciągania najbliższych elementów, mamy $O(n^2 \log(n))$, ale kosztuje dodatkowe $O(n)$ pamięci
- istnieją algorytmy dla single i complete linkage o złożoności $O(n^2)$

Klasteryzacja hierarchiczna - zastosowania

- szczególnie dobre, gdy ważne są **konkretne próbki**
- możemy wtedy dokładnie **analizować dendrogramy**, żeby zrozumieć relacje między próbkami i ich grupami
- koszt obliczeniowy jest wtedy rozsądnie niski

Klasteryzacja hierarchiczna - zalety i wady

Zalety:

- hierarchia klastrow, dużo dodatkowych informacji
- interpretowalność (dendrogram)
- dla dowolnych metryk

Wady:

- nieskalowalne, bardzo duży koszt obliczeniowy i pamięciowy
- konieczność doboru finalnej liczby klastrow

k-medoids

k-medoids

- **partitioning / prototype-based / centroid-based**
- **idea:** jako prototyp klastra wybierzmy "centralny" punkt ze zbioru
- jest to **medoid**, czyli punkt, do którego jest sumarycznie najbliżej z reszty klastra
- używa tylko odległości między punktami ze zbioru, a więc **nie ma funkcji kosztu** jak k-means, więc pozwala na **dowolną metrykę**
- używa algorytmu Partitioning Around Medoids (PAM), lub jego nowoczesnych wersji:
 - FastPAM
 - FasterPAM

k-medoids - inicjalizacja

- **losowo:**
 - najszybsze
 - dobre przy nowszych wariantach (np. FasterPAM)
- **metoda BUILD:**
 - pierwszy punkt to globalny medoid - z najmniejszą sumą odległości z reszty zbioru
 - kolejne medoidy wybiera się tak, żeby jak najbardziej zmniejszyły sumę odległości od reszty zbioru do medoidów
 - poprawia jakość klasteryzacji i szybkość zbieżności, ale ma złożoność $O(n^2k)$

Partitioning Around Medoids (PAM) - algorytm

1. **Faza BUILD:** zainicjalizuj medoidy
2. **Faza SWAP:** powtarzaj do osiągnięcia zbieżności:
 - a. Idź po kolejnych klastrach, a dla każdego klastra po punktach z tego klastra
 - b. Sprawdź, czy zmiana obecnego medoidu z klastra na dane punkt zmniejszy globalną sumę odległości (od punktu do najbliższego medoidu):
 - i. jeżeli tak, to zapamiętaj punkt i koszt
 - ii. jeżeli nie, to idź dalej
 - c. Dokonaj najbardziej opłacalnej zmiany
 - d. Przypisz punkty do najbliższego medoidu

k-medoids - złożoność obliczeniowa

- w każdej iteracji rozważamy de facto **wszystkie możliwości**, odległości każdy-z-każdym
- daje to **złożoność pojedynczej iteracji**:
 - czasową $O(k(n - k)^2)$
 - pamięciową $O(n^2)$
- zwykle **wystarcza mało iteracji** dzięki metodzie BUILD, ale koszt i tak jest duży
- fazę SWAP można wydajnie przyspieszyć, redukując liczbę rozważanych punktów, jak w FastPAM i FasterPAM
- **predykcja** jest analogiczna do k-means

k-medoids

- **bardziej odporny na szum i outliery** niż k-means, bo wymaga konkretnego punktu, który mniej się "przesuwa" niż centroid
- obliczamy explicite **odległości między punktami**:
 - dowolna metryka
 - dużo większa złożoność obliczeniowa i pamięciowa
- w praktyce działa dla max kilkunastu tysięcy punktów
- istnieją bardziej skalowalne algorytmy, wykorzystujące próbkowanie części danych w połączeniu z PAM, np. CLARA, CLARANS

k-medoids - zalety i wady

Zalety:

- prostota
- dowolne metryki
- mniej czułe na outliery od k-means
- doskonale uwspółbieżnialne
- interpretowalność (medoidy)
- potrafi robić predykcję

Wady:

- prowadzi do sferycznych klastrów
- ciężko dobrać liczbę klastrów
- duży koszt obliczeniowy i pamięciowy